MODULO 03 · ANGULAR LABORATORIO

Servizi: il registro degli studenti

Il progetto registro-studenti: un servizio condiviso da più pagine, una tabella filtrabile, un form di inserimento e il dettaglio con parametro

Classe: Quarta Informatica **Durata:** 4 ore **Versione:** settembre 2026

OBIETTIVI

Al termine di questo laboratorio saprai:

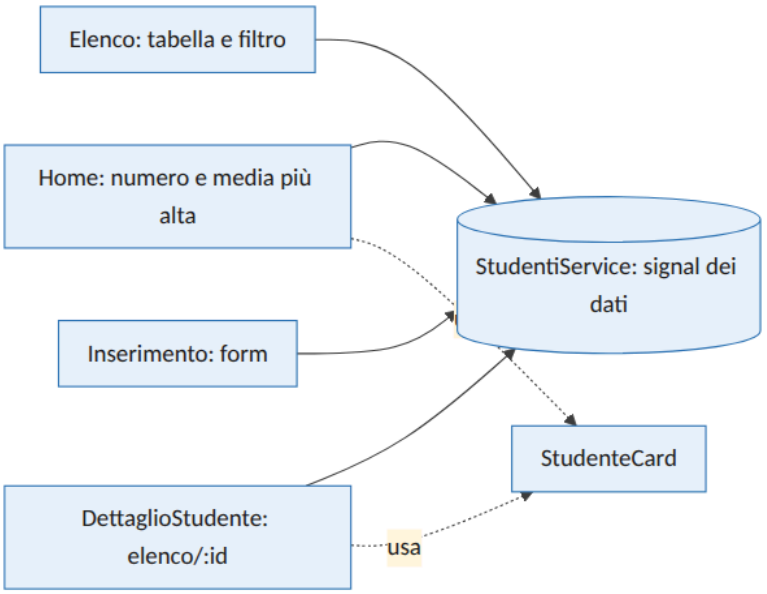
- creare un servizio con `@Service()` che tiene i dati in un signal privato e li espone in sola lettura;
- usare lo stesso servizio da più pagine con `inject()`;
- filtrare una tabella con `[(ngModel)]` e `computed`, e colorare i valori con `[class.nome]`;
- scrivere un form con validazione e tornare all'elenco con `router.navigate()` dopo il salvataggio;
- riusare un componente figlio con input e output in due pagine diverse;
- leggere un parametro dall'URL con `paramMap` e passare allo studente successivo.

PREREQUISITI

Dispense 03.11 (routing) e 03.13 (servizi); laboratori 03.08 (Bootstrap) e 03.12 (routing).

Prima di iniziare

Costruirai un piccolo registro: una home con qualche numero, l'elenco degli studenti in una tabella filtrabile, un form per inserirne di nuovi e una scheda di dettaglio. Tutte le pagine lavorano sugli **stessi dati**, custoditi da un servizio.



Crea su GitHub il repository **`cognome-nome-registro-studenti`** con README, aprilo in un codespace (o clonalo sul PC) e nella radice:

```
ng new registro-studenti     # stylesheet: CSS, SSR: No
cd registro-studenti
npm install bootstrap@5.3.8
```

Registra Bootstrap in `angular.json` come nel laboratorio 03.08, avvia `ng serve` e tieni un secondo terminale dentro `registro-studenti` per i comandi della CLI.

Passo 1 — Il modello Studente

```
ng g interface modelli/studente
```

`src/app/modelli/studente.ts`

```
export interface Studente {
  id: number;
  nome: string;
  classe: string;
  mediaVoti: number;
}
```

Un'interfaccia basta: uno studente è un insieme di dati senza comportamento, come quelli che un giorno arriveranno da un server.

Risultato atteso: il file esiste e il terminale di `ng serve` non segnala errori. Fai il commit `Passo 1: modello Studente`.

Passo 2 — Il servizio StudentiService

```
ng g s services/studenti-service
```

`src/app/services/studenti-service.ts`

```
import { Service, computed, signal } from '@angular/core';
import { Studente } from '../modelli/studente';

@Service()
export class StudentiService {
  private readonly dati = signal<Studente[]>([
    { id: 1, nome: 'Luca Bianchi', classe: '4B', mediaVoti: 7.5 },
    { id: 2, nome: 'Sara Rossi', classe: '4B', mediaVoti: 5.8 },
    { id: 3, nome: 'Marco Verdi', classe: '4A', mediaVoti: 8.2 },
    { id: 4, nome: 'Giulia Neri', classe: '4A', mediaVoti: 6.4 },
    { id: 5, nome: 'Paolo Russo', classe: '4B', mediaVoti: 4.9 },
  ]);

  readonly studenti = this.dati.asReadonly();
  readonly numero = computed(() => this.dati().length);
  readonly migliore = computed(() => {
    const lista = this.dati();
    if (lista.length === 0) {
      return undefined;
    }
    return lista.reduce((m, s) => (s.mediaVoti > m.mediaVoti ? s : m));
  });

  aggiungi(nome: string, classe: string, mediaVoti: number): Studente {
    const nuovo: Studente = { id: this.prossimoId(), nome: nome, classe: classe, mediaVoti:
mediaVoti };
    this.dati.update(lista => [...lista, nuovo]);
    return nuovo;
  }

  rimuovi(id: number): void {
    this.dati.update(lista => lista.filter(s => s.id !== id));
  }

  trova(id: number): Studente | undefined {
    return this.dati().find(s => s.id === id);
  }

  successivo(id: number): Studente | undefined {
    const lista = this.dati();
    const i = lista.findIndex(s => s.id === id);
    if (i === -1) {
      return undefined;
    }
    return lista[(i + 1) % lista.length];
  }

  private prossimoId(): number {
    return Math.max(0, ...this.dati().map(s => s.id)) + 1;
  }
}
```

Leggi il servizio con attenzione, perché tutto il resto del laboratorio lo usa:

- `dati` è il signal con l'elenco ed è **privato**: solo il servizio può modificarlo. I componenti leggono `studenti`, la sua versione in sola lettura ottenuta con `asReadonly()`, e cambiano i dati solo attraverso i metodi.
- `numero` e `migliore` sono valori derivati. `reduce` scorre la lista tenendo lo studente con la media più alta trovato fin qui.

- `aggiungi` calcola da solo l'**id**: il più grande tra quelli esistenti, più uno. Non basta "numero di studenti + 1": dopo aver eliminato lo studente 2, il sesto inserito avrebbe l'id 5, già usato.
- `trova` restituisce lo studente oppure `undefined`; `successivo` restituisce quello dopo nell'elenco e, dopo l'ultimo, ricomincia dal primo grazie al resto della divisione (`%`).

Risultato atteso: nessun errore di compilazione. Fai il commit `Passo 2: servizio StudentiService`.

Passo 3 — Le pagine, le route e il menu

```
ng g c home
ng g c elenco
ng g c inserimento
```

`src/app/app.routes.ts`

```ts
import { Routes } from '@angular/router';
import { Home } from './home/home';
import { Elenco } from './elenco/elenco';
import { Inserimento } from './inserimento/inserimento';

export const routes: Routes = [
  { path: '', redirectTo: 'home', pathMatch: 'full' },
  { path: 'home', component: Home },
  { path: 'elenco', component: Elenco },
  { path: 'inserimento', component: Inserimento },
  { path: '**', redirectTo: 'home' },
];
```

`src/app/app.ts`

```ts
import { Component } from '@angular/core';
import { RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';

@Component({
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  selector: 'app-root',
  styleUrl: './app.css',
  templateUrl: './app.html',
})
export class App {}
```

`src/app/app.html`

```
<nav class="navbar navbar-expand bg-primary mb-4" data-bs-theme="dark">
  <div class="container">
    <span class="navbar-brand">Registro</span>
    <ul class="navbar-nav">
      <li class="nav-item">
        <a class="nav-link" routerLink="/home" routerLinkActive="active">Home</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" routerLink="/elenco" routerLinkActive="active">Elenco</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" routerLink="/inserimento" routerLinkActive="active">Nuovo
studente</a>
      </li>
    </ul>
  </div>
</nav>

<main class="container pb-4">
  <router-outlet />
</main>
```

La home usa il servizio per mostrare il numero di studenti. Il riquadro grande in cima è il modo di Bootstrap 5 per ottenere l'effetto del vecchio "jumbotron":

`src/app/home/home.ts`

```
import { Component, inject } from '@angular/core';
import { RouterLink } from '@angular/router';
import { StudentiService } from '../services/studenti-service';

@Component({
  imports: [RouterLink],
  selector: 'app-home',
  styleUrl: './home.css',
  templateUrl: './home.html',
})
export class Home {
  readonly studentiService = inject(StudentiService);
}
```

`src/app/home/home.html`

```
<div class="p-5 mb-4 bg-body-tertiary rounded-3">
  <h1>Registro degli studenti</h1>
  <p class="lead">Studenti registrati: {{ studentiService.numero() }}</p>
  <a class="btn btn-primary" routerLink="/elenco">Vai all'elenco</a>
  <a class="btn btn-outline-primary ms-2" routerLink="/inserimento">Nuovo studente</a>
</div>
```

La proprietà `studentiService` non è `private` perché il template la usa.

Risultato atteso: barra blu con tre voci; la home dice "Studenti registrati: 5"; le altre due pagine mostrano per ora il testo generato ("elenco works!"). Fai il commit `Passo 3: pagine e route`.

Passo 4 — L'elenco in una tabella

`src/app/elenco/elenco.ts`

```ts
import { Component, inject } from '@angular/core';
import { StudentiService } from '../services/studenti-service';

@Component({
  imports: [],
  selector: 'app-elenco',
  styleUrl: './elenco.css',
  templateUrl: './elenco.html',
})
export class Elenco {
  private readonly studentiService = inject(StudentiService);
  readonly studenti = this.studentiService.studenti;

  rimuovi(id: number): void {
    this.studentiService.rimuovi(id);
  }
}
```

`src/app/elenco/elenco.html`

```html
<h2>Elenco degli studenti</h2>

<table class="table table-striped table-hover">
  <thead>
    <tr><th>Id</th><th>Nome</th><th>Classe</th><th>Media</th><th></th></tr>
  </thead>
  <tbody>
    @for (s of studenti(); track s.id) {
      <tr>
        <td>{{ s.id }}</td>
        <td>{{ s.nome }}</td>
        <td>{{ s.classe }}</td>
        <td [class.text-success]="s.mediaVoti >= 6" [class.text-danger]="s.mediaVoti < 6">
{{ s.mediaVoti }}</td>
        <td class="text-end">
          <button type="button" class="btn btn-sm btn-outline-danger"
(click)="rimuovi(s.id)">Elimina</button>
        </td>
      </tr>
    }
  </tbody>
</table>
```

Osserva:

- `readonly studenti = this.studentiService.studenti;` rende disponibile al template il signal del servizio. Non è una copia: è lo stesso signal, quindi quando il servizio cambia i dati, la tabella si aggiorna.
- `[class.text-success]` e `[class.text-danger]` accendono l'una o l'altra classe di Bootstrap in base alla media: verde da 6 in su, rosso sotto.
- **Elimina** chiama un metodo del componente, che a sua volta chiama il servizio: il componente non tocca mai i dati direttamente.

Risultato atteso: una tabella a righe alterne con cinque studenti; le medie di Sara Rossi e Paolo Russo sono rosse, le altre verdi. Elimina uno studente, poi vai sulla home: il numero è sceso a 4, perché le due pagine leggono lo stesso servizio. Fai il commit `Passo 4: tabella degli studenti`.

Passo 5 — Il filtro per classe

Sopra la tabella aggiungi un campo: scrivendo `4A` restano solo gli studenti di quella classe. Il testo del filtro è un signal collegato al campo con `[(ngModel)]`; l'elenco filtrato è un `computed`.

`src/app/elenco/elenco.ts`

```ts
import { Component, computed, inject, signal } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { StudentiService } from '../services/studenti-service';

@Component({
  imports: [FormsModule],
  selector: 'app-elenco',
  styleUrl: './elenco.css',
  templateUrl: './elenco.html',
})
export class Elenco {
  private readonly studentiService = inject(StudentiService);
  readonly filtroClasse = signal('');
  readonly filtrati = computed(() => {
    const filtro = this.filtroClasse().trim().toUpperCase();
    const tutti = this.studentiService.studenti();
    if (filtro === '') {
      return tutti;
    }
    return tutti.filter(s => s.classe.toUpperCase() === filtro);
  });

  rimuovi(id: number): void {
    this.studentiService.rimuovi(id);
  }
}
```

Nel template aggiungi il campo dopo il titolo e racchiudi la tabella in un `@if`; il ciclo ora usa `filtrati()`:

`src/app/elenco/elenco.html`

```
<h2>Elenco degli studenti</h2>

<div class="row g-2 align-items-center mb-3">
  <div class="col-auto">
    <label for="filtro" class="col-form-label">Filtra per classe</label>
  </div>
  <div class="col-auto">
    <input id="filtro" class="form-control" [(ngModel)]="filtroClasse" placeholder="es.
4B">
  </div>
</div>

@if (filtrati().length === 0) {
  <div class="alert alert-warning">
    @if (filtroClasse()) {
      Nessuno studente trovato per la classe {{ filtroClasse() }}
    } @else {
      Il registro è vuoto.
    }
  </div>
} @else {
  <table class="table table-striped table-hover">
    <thead>
      <tr><th>Id</th><th>Nome</th><th>Classe</th><th>Media</th><th></th></tr>
    </thead>
    <tbody>
      @for (s of filtrati(); track s.id) {
        <tr>
          <td>{{ s.id }}</td>
          <td>{{ s.nome }}</td>
          <td>{{ s.classe }}</td>
          <td [class.text-success]="s.mediaVoti >= 6" [class.text-danger]="s.mediaVoti <
6">{{ s.mediaVoti }}</td>
          <td class="text-end">
            <button type="button" class="btn btn-sm btn-outline-danger"
(click)="rimuovi(s.id)">Elimina</button>
          </td>
        </tr>
      }
    </tbody>
  </table>
}
```

A ogni tasto premuto `[(ngModel)]` aggiorna il signal `filtroClasse`; il `computed` `filtrati`, che lo legge, viene ricalcolato e la tabella cambia. `trim()` e `toUpperCase()` rendono il filtro tollerante: `4a` e `4A` trovano gli stessi studenti. Il campo non sta dentro un `<form>`, quindi non serve l'attributo `name`.

Risultato atteso: con `4a` restano Marco Verdi e Giulia Neri; con `5C` compare l'avviso giallo "Nessuno studente trovato per la classe 5C"; cancellando il filtro ritornano tutti. Fai il commit `Passo 5: filtro per classe`.

Passo 6 — La pagina di inserimento

`src/app/inserimento/inserimento.ts`

```
import { Component, inject, signal } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { StudentiService } from '../services/studenti-service';

@Component({
  imports: [FormsModule],
  selector: 'app-inserimento',
  styleUrl: './inserimento.css',
  templateUrl: './inserimento.html',
})
export class Inserimento {
  private readonly studentiService = inject(StudentiService);
  private readonly router = inject(Router);
  readonly nome = signal('');
  readonly classe = signal('4B');
  readonly media = signal(6);

  salva(): void {
    this.studentiService.aggiungi(this.nome().trim(), this.classe().trim().toUpperCase(),
Number(this.media()));
    this.router.navigate(['/elenco']);
  }
}
```

`src/app/inserimento/inserimento.html`

```
<h2>Nuovo studente</h2>

<form #modulo="ngForm" (ngSubmit)="salva()" class="p-4 bg-body-tertiary rounded-3">
  <div class="mb-3">
    <label for="nome" class="form-label">Nome e cognome</label>
    <input id="nome" name="nome" class="form-control" [(ngModel)]="nome" required>
  </div>
  <div class="mb-3">
    <label for="classe" class="form-label">Classe</label>
    <input id="classe" name="classe" class="form-control" [(ngModel)]="classe" required>
  </div>
  <div class="mb-3">
    <label for="media" class="form-label">Media dei voti</label>
    <input id="media" name="media" type="number" class="form-control" [(ngModel)]="media"
           required min="1" max="10" step="0.1">
  </div>
  <button type="submit" class="btn btn-primary" [disabled]="modulo.invalid">Salva</button>
</form>
```

Le novità:

- Dentro un `<form>`, ogni campo con `[(ngModel)]` deve avere l'attributo `name`: Angular lo usa per registrare il campo nel form.
- `#modulo="ngForm"` crea una variabile di template che punta non all'elemento `<form>`, ma all'oggetto con cui Angular controlla il form. `modulo.invalid` è vero quando almeno un campo non rispetta le sue regole: `required` (obbligatorio), `min` e `max` (valori ammessi per un campo numerico). Il pulsante resta disabilitato finché il form non è valido.
- `(ngSubmit)` scatta sia con il pulsante `type="submit"` sia premendo `Invio` in un campo, senza ricaricare la pagina.

- Dopo il salvataggio, `router.navigate(['/elenco'])` porta all'elenco **da codice**: non c'è un link da cliccare, la navigazione è la conseguenza di un'azione.
- `Number(this.media())` garantisce che la media sia un numero; la classe viene scritta in maiuscolo per coerenza con le altre.

Risultato atteso: con il nome vuoto o una media di 11 il pulsante **Salva** è grigio; con dati validi, **Salva** porta all'elenco, dove il nuovo studente compare in fondo con l'id 6. Fai il commit `Passo 6: pagina di inserimento`.

Passo 7 — Il componente studente-card

Una scheda riutilizzabile che mostra uno studente, con un pulsante facoltativo per rimuoverlo. Il pulsante non rimuove niente da solo: **avvisa** il padre con un output, e sarà il padre a decidere che cosa fare.

```
ng g c studente-card
```

`src/app/studente-card/studente-card.ts`

```
import { Component, input, output } from '@angular/core';
import { Studente } from '../modelli/studente';

@Component({
  imports: [],
  selector: 'app-studente-card',
  styleUrl: './studente-card.css',
  templateUrl: './studente-card.html',
  host: { class: 'card' },
})
export class StudenteCard {
  readonly studente = input.required<Studente>();
  readonly azioni = input(true);
  readonly rimosso = output<Studente>();

  rimuovi(): void {
    this.rimosso.emit(this.studente());
  }
}
```

`src/app/studente-card/studente-card.html`

```
<div class="card-body">
  <h5 class="card-title">{{ studente().nome }}</h5>
  <p class="card-text">
    Classe {{ studente().classe }}, media
    <strong [class.text-success]="studente().mediaVoti >= 6" [class.text-
danger]="studente().mediaVoti < 6">
      {{ studente().mediaVoti }}
    </strong>
  </p>
  @if (azioni()) {
    <button type="button" class="btn btn-sm btn-outline-danger" (click)="rimuovi()">Rimuovi
dal registro</button>
  }
</div>
```

`studente` è obbligatorio; `azioni` è facoltativo e vale `true` se il padre non lo passa. Il primo uso è nella home, per mostrare lo studente con la media più alta, **senza** pulsante. Aggiungi `StudenteCard` agli `imports` di `Home` e in fondo a `home.html`:

```
@if (studentiService.migliore(); as m) {
  <h2 class="h4">La media più alta</h2>
  <app-studente-card [studente]="m" [azioni]="false" />
}
```

`[azioni]="false"` richiede le quadre: senza, il figlio riceverebbe la stringa `"false"`, e la compilazione la rifiuterebbe perché l'input è booleano.

Risultato atteso: nella home, sotto il riquadro, la scheda di Marco Verdi con la media 8.2 in verde e nessun pulsante. Fai il commit `Passo 7: componente studente-card`.

Passo 8 — Il dettaglio con il parametro nell'URL

```
ng g c dettaglio-studente
```

Aggiungi la route prima della wildcard (con l'import di `DettaglioStudente` in cima al file):

```
  { path: 'elenco/:id', component: DettaglioStudente },
```

`src/app/dettaglio-studente/dettaglio-studente.ts`

```
import { Component, OnInit, inject, signal } from '@angular/core';
import { ActivatedRoute, Router, RouterLink } from '@angular/router';
import { Studente } from '../modelli/studente';
import { StudentiService } from '../services/studenti-service';
import { StudenteCard } from '../studente-card/studente-card';

@Component({
  imports: [RouterLink, StudenteCard],
  selector: 'app-dettaglio-studente',
  styleUrl: './dettaglio-studente.css',
  templateUrl: './dettaglio-studente.html',
})
export class DettaglioStudente implements OnInit {
  private readonly route = inject(ActivatedRoute);
  private readonly router = inject(Router);
  private readonly studentiService = inject(StudentiService);
  readonly studente = signal<Studente | undefined>(undefined);

  ngOnInit(): void {
    this.route.paramMap.subscribe(parametri => {
      const id = Number(parametri.get('id'));
      const trovato = this.studentiService.trova(id);
      if (trovato) {
        this.studente.set(trovato);
      } else {
        this.router.navigate(['/elenco']);
      }
    });
  }

  successivo(): void {
    const attuale = this.studente();
    if (!attuale) {
      return;
    }
    const prossimo = this.studentiService.successivo(attuale.id);
    if (prossimo) {
      this.router.navigate(['/elenco', prossimo.id]);
    }
  }

  rimuovi(s: Studente): void {
    this.studentiService.rimuovi(s.id);
    this.router.navigate(['/elenco']);
  }
}
```

`src/app/dettaglio-studente/dettaglio-studente.html`

```
@if (studente(); as s) {
  <h2>Scheda dello studente</h2>
  <app-studente-card [studente]="s" (rimosso)="rimuovi($event)" />
  <div class="mt-3">
    <a class="btn btn-outline-secondary" routerLink="/elenco">Torna all'elenco</a>
    <button type="button" class="btn btn-primary ms-2" (click)="successivo()">Studente
successivo</button>
  </div>
}
```

Infine, nell'elenco, aggiungi a ogni riga il link al dettaglio (e `RouterLink` agli `imports` di `Elenco`); l'ultima cella diventa:

```
          <td class="text-end">
            <a class="btn btn-sm btn-outline-primary" [routerLink]="['/elenco',
s.id]">Dettaglio</a>
            <button type="button" class="btn btn-sm btn-outline-danger ms-1"
(click)="rimuovi(s.id)">Elimina</button>
          </td>
```

Tre punti chiave:

- L'`id` arriva dall'URL come **stringa**: `Number(...)` lo converte, altrimenti `trova` confronterebbe `'2'` con `2` e non troverebbe nessuno.
- **Studente successivo** porta da `/elenco/2` a `/elenco/3`: la route è la stessa e Angular riusa il componente, senza eseguire di nuovo `ngOnInit`. Solo perché il parametro è letto con `paramMap.subscribe` la pagina si aggiorna. Con lo snapshot (`this.route.snapshot.paramMap.get('id')`) l'URL cambierebbe ma la scheda resterebbe ferma.
- Se lo studente non esiste (per esempio `/elenco/99`), il componente rimanda all'elenco. Lo stesso succede dopo la rimozione: la card emette `rimosso`, il padre chiama il servizio e poi naviga.

Risultato atteso: **Dettaglio** apre la scheda dello studente; **Studente successivo** scorre gli studenti e dopo l'ultimo ricomincia dal primo; **Rimuovi dal registro** elimina lo studente e torna all'elenco; `/elenco/99` porta all'elenco. Fai il commit `Passo 8: dettaglio con parametro` e il push.

Prova tu

PROVA TU

(livello 1) **Media della classe.** In `Elenco` aggiungi un `computed` `mediaClasse` con la media dei voti degli studenti **filtrati** (attenzione alla divisione per zero se la lista è vuota) e mostrala sotto la tabella con due decimali (`toFixed(2)`). Filtrando `4A` deve comparire 7.30.

PROVA TU

(livello 2) **Ordinamento.** Aggiungi all'elenco due pulsanti, **Per nome** e **Per media**, che impostano un signal `ordine`. Un `computed` `visibili` restituisce una **copia** di `filtrati()` ordinata per nome (`a.nome.localeCompare(b.nome)`) oppure per media decrescente; il `@for` usa `visibili()`.

PROVA TU

(livello 3) **Modifica.** Aggiungi al servizio un metodo `aggiorna(modificato: Studente)` che sostituisce lo studente con lo stesso `id` usando `map`. Crea il componente `modifica-studente` con la route `modifica/:id`: in `ngOnInit` legge l'`id`, trova lo studente e ne copia i dati nei signal del form (lo stesso form dell'inserimento); **Salva** chiama `aggiorna` e naviga al dettaglio. Aggiungi nell'elenco il link **Modifica** con `[routerLink]="['/modifica', s.id]"`. Qui lo snapshot va bene: perché?

Errori comuni

Messaggio o sintomo	Causa	Soluzione
`NG8002: Can't bind to 'ngModel' since it isn't a known property of 'input'`	Manca `FormsModule` negli `imports` del componente	Importa `FormsModule` da `@angular/forms`
Console: `NG01352: If ngModel is used within a form tag, either the name attribute must be set ...`	Un campo con `[(ngModel)]` dentro il `<form>` non ha `name`	Aggiungi `name="..."` a ogni campo del form
`NG2028: @Service class cannot use constructor dependency injection.`	Nel servizio hai scritto `constructor(private router: Router)`	Nei servizi `@Service()` usa `inject()` nelle proprietà
Console: `NG0201: No provider found for` seguito dal nome del servizio	Il servizio è decorato con `@Injectable()` senza `providedIn: 'root'`	Usa `@Service()` oppure `@Injectable({ providedIn: 'root' })`
Il dettaglio rimanda sempre all'elenco	L'`id` letto dall'URL è una stringa e `trova` non lo trova	Converti con `Number(...)`
Studente successivo cambia l'URL ma non la scheda	Hai letto il parametro con lo snapshot, oppure `studente` non è un signal	Usa `paramMap.subscribe` e un signal
Dopo alcune eliminazioni due studenti hanno lo stesso id	L'id è calcolato come "numero di studenti + 1"	Usa il massimo degli id esistenti + 1
Il filtro `4b` non trova nessuno	Il confronto distingue maiuscole e minuscole	Confronta dopo `toUpperCase()`
Ricaricando la pagina gli studenti aggiunti spariscono	Il servizio vive in memoria: è il comportamento previsto	Per conservarli serviranno `localStorage` o un server

Domande a cui rispondere

Scrivi le risposte nel file `RISPOSTE.md` nella radice del repository.

1. Che cos'è `app.routes.ts` e che cosa contiene in questo progetto? Elenca gli URL e il componente che mostrano.
2. Che cos'è un service e perché lo si usa? Perché gli studenti stanno in `StudentiService` e non nel componente `Elenco`?
3. Come si legge l'`id` dello studente dall'URL? Perché va convertito con `Number`?
4. Perché il signal `dati` è privato e i componenti leggono `studenti`? Che cosa potrebbe succedere se `Elenco` potesse chiamare `set` sul signal?
5. Come viene calcolato l'id di un nuovo studente? Mostra con un esempio perché "numero di studenti + 1" sarebbe sbagliato.
6. Che cosa succede, passo per passo, quando scrivi una lettera nel campo del filtro?
7. A che cosa servono `#modulo="ngForm"` e `[disabled]="modulo.invalid"`? Quali condizioni rendono il form non valido?
8. Perché dopo il salvataggio si usa `router.navigate(['/elenco'])` invece di un link?

9. Quali input e quale output ha `StudenteCard`? Perché la card non chiama direttamente il servizio per rimuovere lo studente?
10. Perché **Studente successivo** funziona solo con `paramMap.subscribe`? Che cosa vedresti con lo snapshot?

> **CONSEGNA**
>
> Su Classroom consegna il link al repository `cognome-nome-registro-studenti` con il progetto `registro-studenti`, i commit dei passi (da `Passo 1: modello Studente` a `Passo 8: dettaglio con parametro`), almeno una prova in autonomia (commit `Prova tu livello N`) e il file `RISPOSTE.md` con le dieci risposte.

Griglia di valutazione

Criterio	Punti
Modello e servizio: signal privato, `asReadonly`, metodi, id automatico (Passi 1-2)	1,5
Route, menu con voce attiva e home che legge il servizio (Passo 3)	1
Tabella con media colorata e rimozione (Passo 4)	1
Filtro per classe con `[(ngModel)]`, `computed` e messaggio (Passo 5)	1
Form di inserimento con validazione e navigazione da codice (Passo 6)	1,5
Componente `studente-card` con input e output, usato in due pagine (Passo 7)	1
Dettaglio con parametro, studente successivo e redirect (Passo 8)	1,5
Almeno una prova in autonomia funzionante	0,5
Un commit per passo con messaggi chiari; `RISPOSTE.md` completo	1
Totale	**10**